

AI Fastener Inspection and Sorting System

Project File Guide

What every code file does, how the files work together, and where to look for each feature

This software looks at a fastener through a camera, decides whether it is a **bolt, nut, screw or washer**, measures it, checks the size against standard tolerances, and tells an ESP32 board which sorting bin to drop it in. It is a desktop application written in Python, with the windows built in PySide6 (Qt).

The guide is produced from the header comment at the top of each file and the section headings inside the code, so the names here are the names you will find when you open the file.

How to read this guide

- Section 1 shows the path one inspection takes through the code.
- Section 2 lists every file with a one-line description, and what else is in the project folder.
- Section 3 describes each file in detail: what it does, its main parts, the sections inside it, and which other files it works with.
- Section 4 covers the ESP32 firmware, and Section 5 how to run everything.

Generated on 09 October 2026 from the project source code.

1. How one inspection flows through the code

When the operator presses **INSPECT**, these steps run in order. Steps 2 to 6 run in a background thread so the window does not freeze.

Step	What happens	File that does it
1	The picture comes from the live camera, an opened file, or a built-in sample.	frontend/main_window.py backend/camera_capture.py
2	The inspection is started and coordinated.	backend/inspection_pipeline.py
3	CLASSIFY: decide what kind of fastener it is. One of three engines is used, chosen in Settings: the trained YOLO model, OpenCV shape rules, or Google Gemini.	backend/yolo_classifier.py backend/opencv_shape_classifier.py backend/gemini_cloud_classifier.py
4	MEASURE: find the part's outline and convert pixels to millimetres using the camera scale. Refuses to measure if the outline is not a clean, whole part.	backend/opencv_measurement.py
5	DECIDE: match the nearest standard size, check tolerances, give ACCEPT / REINSPECT / REJECT, and choose the bin and chute angle. In 'Sort by Type Only' mode the size is shown but not judged.	backend/tolerance_and_bin_decision.py
6	SAVE: store the result in the inspection history.	backend/sqlite_database.py
7	SORT: send commands to the ESP32 (chute, pad tilt, conveyor). Without hardware, a built-in simulator answers.	backend/esp32_communication.py firmware_esp32/firmware_esp32.ino
8	SHOW: update the result panel, the picture overlay, the history table and the bin fill levels.	frontend/inspection_result_panel.py frontend/main_window.py

The six tabs and their files

Tab in the app	File
1 Live Inspection Workstation	frontend/main_window.py + frontend/inspection_result_panel.py
2 Custom Bins & Chute Angles	frontend/tab_bins_and_chute_angles.py
3 Hardware & Sorting Chute	frontend/tab_hardware_control.py
4 ISO Specifications	frontend/tab_iso_specifications.py
5 Camera Calibration	frontend/tab_camera_calibration.py
6 Quality Audit History	frontend/tab_inspection_history.py
Settings button (header)	frontend/ai_settings_dialog.py

2. Every file at a glance

Start	What it does
main.py	Starts the application.

Backend - everything that is not a window or a button	What it does
backend/app_config.py	Settings and constants used everywhere.
backend/app_logging.py	Sets up the two log files.
backend/sqlite_database.py	All reading and writing of the database.
backend/camera_capture.py	Live video from the camera.
backend/image_loading_and_overlays.py	Loads images and draws boxes and labels on them.
backend/yolo_classifier.py	Names the part with the trained YOLO model.
backend/opencv_shape_classifier.py	Names the part with OpenCV shape rules (no training).
backend/gemini_cloud_classifier.py	Names the part with Google Gemini (optional, online).
backend/opencv_measurement.py	Measures the part in millimetres.
backend/tolerance_and_bin_decision.py	Checks the size and chooses the bin.
backend/inspection_pipeline.py	Runs one whole inspection, step by step.
backend/esp32_communication.py	Sends commands to the ESP32 that moves the machine.

Frontend - windows, tabs and buttons	What it does
frontend/main_window.py	The main window and Tab 1 (Live Inspection).
frontend/inspection_result_panel.py	The result panel on the right of Tab 1.
frontend/tab_bins_and_chute_angles.py	Tab 2: sorting bins and chute angles.
frontend/tab_hardware_control.py	Tab 3: ESP32 connection and manual motor tests.
frontend/tab_iso_specifications.py	Tab 4: standard sizes and tolerances.
frontend/tab_camera_calibration.py	Tab 5: camera scale (mm per pixel).
frontend/tab_inspection_history.py	Tab 6: history of inspected parts.
frontend/ai_settings_dialog.py	Settings window: which AI model to use.
frontend/app_stylesheet.py	Colours, fonts and button styles.

Tools - run from the command line	What it does
build_classifier_dataset.py	Turns the downloaded photos into training images.
train_yolo_classifier.py	Trains and tests the YOLO model.
run_all_tests.py	Checks that the whole system still works.

Firmware	What it does
firmware_esp32/firmware_esp32.ino	The program that runs on the ESP32 board.

Other files and folders	What they are
models/	The trained YOLO model (fastener_yolo11n_cls.pt), its class list and its accuracy report.
FASTENER_SORTER.v3i.yolov11/	The downloaded Roboflow dataset: photos with a box drawn round each part.
fastener_sorter_cls/	Training images built from it by build_classifier_dataset.py, one folder per class.
my_camera_photos/	Photos saved with the app's 'Save Photo for Training' button (created on first save).
training_runs/	Output of the last training run.
sample_images/	The four drawn demo fasteners used by the 'Load Sample' menu.
frontend/assets/	Small icons used by the stylesheet.
fastener_inspection.db	The SQLite database: specifications, bins, history, calibration, settings.
logs.txt and logs/	Step-by-step session log, and the detailed technical log.
requirements.txt	The Python packages the project needs.
.env / .env.example	The Gemini API key (private, not shared) and a template for it.
README.md, PROJECT_SYNOPSIS.txt, dataset_overview.txt, lib_tool.txt	Project documents.

3. Each file in detail

Start

main.py

The starting point of the application. Run this file to open the desktop app:

```
python main.py
```

MAIN PARTS

- Environment check: restarts under the project's own Python (.venv) when started with another one
- Crash logging: any unexpected error is written to the log instead of vanishing
- main(): creates the Qt application, applies the stylesheet and shows the main window

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **ENVIRONMENT CHECK** - The YOLO model needs the packages installed in the project's .venv folder. A plain "python main.py" may start a different Python that lacks them, so the app restarts itself there.
- **CRASH LOGGING** - Sends any uncaught error to the application log.
- **START THE APP**

Uses: backend/app_config.py, backend/app_logging.py, frontend/main_window.py

Backend - everything that is not a window or a button

backend/app_config.py

Central settings and constants for the whole app. Change a value here and every file that uses it follows.

MAIN PARTS

- Folder paths (project, logs, sample images, .env)
- App name and version
- The fastener categories (NUT, BOLT, SCREW, WASHER, UNKNOWN) with descriptions and colours
- Confidence thresholds (high / medium / low)
- List of selectable AI models and the Gemini API key reader
- YOLO model file location and minimum confidence
- Camera defaults and supported image file types

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **FOLDER PATHS** - Where the project, logs, sample images and the .env file live.
- **APP NAME AND VERSION** - Shown in the window title.
- **FASTENER CATEGORIES** - Category names, their descriptions and display colours.
- **CONFIDENCE LEVELS** - What counts as high / medium / low confidence, and the colour of each.
- **AI MODEL CHOICES AND GEMINI API KEY**
- **YOLO CLASSIFIER** - The trained model file (made by train_yolo_classifier.py) and how sure it must be.
- **CAMERA DEFAULTS**
- **IMAGE FILE TYPES THE APP CAN OPEN**

Used by: backend/app_logging.py, backend/camera_capture.py, backend/gemini_cloud_classifier.py, backend/image_loading_and_overlays.py, backend/inspection_pipeline.py, backend/opencv_measurement.py, backend/opencv_shape_classifier.py, backend/sqlite_database.py, backend/tolerance_and_bin_decision.py, backend/yolo_classifier.py, frontend/ai_settings_dialog.py, frontend/inspection_result_panel.py, frontend/main_window.py, frontend/tab_inspection_history.py, main.py, run_all_tests.py

backend/app_logging.py

Sets up the two log files the app writes while it runs.

MAIN PARTS

- logs/application.log : detailed technical log (also printed to the console)
- logs.txt : short step-by-step history of each session
- app_logger : use app_logger.info(...) / .warning(...) / .error(...)
- log_session_step() : add one line to logs.txt

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **LOG FILE LOCATIONS**
- **APPLICATION LOG** - Detailed technical log: logs/application.log and the console.
- **SESSION LOG** - Short step-by-step history written to logs.txt.

Uses: backend/app_config.py

Used by: backend/camera_capture.py, backend/esp32_communication.py, backend/gemini_cloud_classifier.py, backend/image_loading_and_overlays.py, backend/inspection_pipeline.py, backend/opencv_measurement.py, backend/opencv_shape_classifier.py, backend/sqlite_database.py, backend/tolerance_and_bin_decision.py, backend/yolo_classifier.py, frontend/ai_settings_dialog.py, frontend/main_window.py, frontend/tab_bins_and_chute_angles.py, frontend/tab_camera_calibration.py, frontend/tab_hardware_control.py, frontend/tab_inspection_history.py, frontend/tab_iso_specifications.py, main.py, run_all_tests.py

backend/sqlite_database.py

Everything that is saved between runs lives in fastener_inspection.db (SQLite). This file is the only place that reads or writes that database.

MAIN PARTS

- Table 'specifications' : standard fastener sizes and tolerance limits
- Table 'trays' : sorting bins (assigned fastener, chute angle, capacity, count)
- Table 'inspection_results' : history of every inspected part
- Table 'calibration' : camera scale in millimetres per pixel
- Table 'machine_settings' : key/value settings (communication mode, COM port, counters...)
- db_instance : the shared database object the rest of the app uses

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **DATABASE FILE LOCATION AND SHARED CONSTANTS**
- **DATABASE CLASS** - Every read and write to the database goes through this class.
 - **SETUP** - Open the database, create the tables, and fill in defaults on first run.
 - **SPECIFICATIONS** - Standard fastener sizes and their tolerance limits.
 - **BATCH COUNTERS** - Totals shown on the inspection tab (accepted, rejected, per category).
 - **SORTING BINS** - Which fastener goes to which bin, plus chute angle, capacity and fill count.
 - **INSPECTION HISTORY** - One saved record per inspected part.
 - **CAMERA CALIBRATION** - The millimetres-per-pixel scale used to convert pixels to real size.
 - **MACHINE SETTINGS** - Key/value settings such as communication mode and COM port.
- **SHARED INSTANCE** - Created once here; every other file imports this same object.

Uses: backend/app_config.py, backend/app_logging.py

Used by: backend/esp32_communication.py, backend/inspection_pipeline.py, backend/opencv_measurement.py, backend/tolerance_and_bin_decision.py, frontend/main_window.py, frontend/tab_bins_and_chute_angles.py, frontend/tab_camera_calibration.py, frontend/tab_hardware_control.py, frontend/tab_inspection_history.py, frontend/tab_iso_specifications.py, run_all_tests.py

backend/camera_capture.py

Reads live video from the inspection camera without freezing the window.

MAIN PARTS

- CameraThread: background thread that grabs frames and hands them to the UI
- scan_available_cameras(): lists the cameras plugged into the PC
- get_preferred_camera_index(): picks an external USB camera before the laptop webcam

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **LIVE CAMERA THREAD** - Grabs frames in the background and sends each one to the UI.
- **CAMERA DISCOVERY** - Find which cameras are plugged in and choose the best one.

Uses: backend/app_config.py, backend/app_logging.py

Used by: frontend/main_window.py

backend/image_loading_and_overlays.py

Helper functions for working with images: loading files, converting them for display in the window, finding the part in the picture, drawing boxes and labels, and drawing the demo samples.

MAIN PARTS

- `load_image()`, `cv_to_qimage()`, `cv_to_qpixmap()`: load files and convert OpenCV images for Qt
- `detect_fastener_roi()`, `centre_guide_box()`: find the part, and the guide box shown on live video
- `draw_classification_overlay()`, `draw_live_scanning_hud()`: drawings on top of the image
- `generate_sample_dataset()`, `render_synthetic_fastener()`: the four built-in 'Load Sample' images

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **LOADING AND QT CONVERSION** - Read image files and convert OpenCV images so the window can show them.
- **FIND THE PART** - Locate the fastener's bounding box in the image.
- **DRAWING ON THE IMAGE** - Result box and label, and the guide shown over live video.
- **SAMPLE IMAGES** - Draw the four built-in demo fasteners used by the 'Load Sample' menu.

Uses: `backend/app_config.py`, `backend/app_logging.py`

Used by: `backend/yolo_classifier.py`, `frontend/main_window.py`, `run_all_tests.py`

backend/yolo_classifier.py

Decides what kind of fastener is in the image using the trained YOLO11n-cls model (`models/fastener_yolo11n_cls.pt`, produced by `train_yolo_classifier.py`). Unlike the OpenCV rules in `opencv_shape_classifier.py`, it has learned from real photos, so it works on ordinary camera pictures and does not need a plain background. It only says WHAT the part is. Size is still measured by `opencv_measurement.py`.

MAIN PARTS

- Model loading: the model is loaded the first time it is needed, not at app start
- Where to look: the whole picture, the centre guide box, and the outline OpenCV finds
- `classify()`: run the model on each of those regions and keep the most confident fastener answer; the model's NO_FASTENER class means there is nothing to identify
- `yolo_classifier`: the shared classifier object the rest of the app uses

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **YOLO CLASSIFIER**
 - **MODEL LOADING** - PyTorch takes a few seconds to load, so it is only loaded on first use.
 - **WHERE TO LOOK** - The part may fill the picture or be a small object in it, so several regions are tried.
 - **CLASSIFY** - Returns the same result format as the other classifiers.
- **SHARED INSTANCE** - Created once here; every other file imports this same object.

Uses: `backend/app_config.py`, `backend/app_logging.py`, `backend/image_loading_and_overlays.py`

Used by: `backend/inspection_pipeline.py`, `frontend/ai_settings_dialog.py`, `frontend/main_window.py`, `run_all_tests.py`

backend/opencv_shape_classifier.py

Decides what kind of fastener is in the image (NUT / BOLT / SCREW / WASHER / UNKNOWN) using only OpenCV shape rules. Works offline, with no internet and no trained model. It expects one part, seen from above, clearly darker or lighter than a plain background.

MAIN PARTS

- Step 1: separate the part from the background
- Step 2: pick the outline most likely to be the fastener
- Step 3: measure shape features (aspect ratio, roundness, hole)
- Step 4: reject shapes that cannot be a single whole fastener
- Step 5: apply the rules -> long + pointed tip = SCREW, long + blunt end = BOLT, compact + round = WASHER, compact + six-sided = NUT

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **OFFLINE SHAPE CLASSIFIER**
 - **STEP 1: SEPARATE THE PART FROM THE BACKGROUND**
 - **STEP 2: PICK THE OUTLINE MOST LIKELY TO BE THE FASTENER**
 - **STEP 3: MEASURE SHAPE FEATURES** (aspect ratio, roundness, hole)
 - **STEP 4: PLAUSIBILITY GATE** - reject shapes that cannot be a single fastener on the pad
 - **STEP 5: CLASSIFICATION RULES** - decide SCREW / BOLT / WASHER / NUT

Uses: backend/app_config.py, backend/app_logging.py

Used by: backend/inspection_pipeline.py, frontend/main_window.py, run_all_tests.py

backend/gemini_cloud_classifier.py

Optional online classifier: sends the image to Google's Gemini model and reads back the category. Needs internet and a GEMINI_API_KEY in the .env file. If it fails, the app falls back to the offline classifier in opencv_shape_classifier.py.

MAIN PARTS

- FASTENER_CLASSIFICATION_PROMPT: the instructions sent to Gemini with every image
- classify_image(): send one image, get category + confidence + reason
- _parse_json_response(): read and validate Gemini's JSON reply
- test_connection(): used by the Settings dialog to check the API key

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **PROMPT** - The instructions sent to Gemini with every image.
- **GEMINI CLIENT**
 - **CLASSIFY** - Send one image to Gemini and get back category, confidence and reason.
 - **RESPONSE CLEAN-UP** - Read Gemini's JSON reply and make sure it is valid.
 - **CONNECTION TEST** - Used by the Settings dialog to check that the API key works.

Uses: backend/app_config.py, backend/app_logging.py

Used by: backend/inspection_pipeline.py, frontend/ai_settings_dialog.py

backend/opencv_measurement.py

Measures a fastener in millimetres from a camera image using OpenCV. It finds the part's outline, measures it in pixels, then converts to mm with the camera calibration.

MAIN PARTS

- Width-profile helpers: separate a bolt/screw outline into head and shank
- Trust check: refuses to measure when the outline found is background, not a part
- outline_has_corners(): tells a round washer outline from a six-sided nut outline
- measure_fastener(): bolt and screw -> length, shank diameter, head width; nut and washer -> hole diameter, outer diameter
- detect_multiple_fasteners(): finds every separate part in one image
- draw_calibrated_overlay(): draws the box, dimension lines and decision badge on the image
- dimension_engine: the shared measurement object the rest of the app uses

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **SHAPE CHECK** - Tells a round outline (washer) from one with flat sides and corners (hex nut).
- **WIDTH-PROFILE HELPERS** - Split a bolt or screw outline into its head and its shank.
- **MEASUREMENT ENGINE** - Turns a camera image into millimetre dimensions.
 - **MEASURE ONE FASTENER** - Find the outline, then measure by type (bolt / nut / washer / screw).
 - **MULTI-FASTENER MODE** - Find every separate part in one image.
 - **RESULT OVERLAY** - Draw the box, dimension lines and decision badge on the image.
- **SHARED INSTANCE** - Created once here; every other file imports this same object.

Uses: backend/app_config.py, backend/app_logging.py, backend/sqlite_database.py

Used by: backend/inspection_pipeline.py, frontend/main_window.py, frontend/tab_camera_calibration.py, run_all_tests.py

backend/tolerance_and_bin_decision.py

Takes the category and the measured dimensions of a part and decides what happens to it: ACCEPT, REINSPECT or REJECT, and which sorting bin and chute angle it goes to.

MAIN PARTS

- `verify_and_decide()`: match the nearest standard size, check each dimension against its tolerance, then decide
- `route_by_type()`: sort by fastener type only, without judging the size (uncalibrated camera)
- Bin routing: exact-size bin -> category bin -> catch-all bin -> reject bin
- `verification_engine`: the shared decision object the rest of the app uses

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **DECISION ENGINE**
 - **MAIN CHECK** - Match the size, check the tolerances, decide ACCEPT / REINSPECT / REJECT.
 - **SORT BY TYPE ONLY** - Used while the camera is not calibrated: the part goes to the bin for its type, the measured size is shown as a guide, and no ACCEPT / REJECT judgement is made on its dimensions.
 - **BIN ROUTING** - Find the bin and chute angle for the part, or fall back to the reject bin.
- **SHARED INSTANCE** - Created once here; every other file imports this same object.

Uses: backend/app_config.py, backend/app_logging.py, backend/sqlite_database.py

Used by: backend/inspection_pipeline.py, frontend/main_window.py, run_all_tests.py

backend/inspection_pipeline.py

Runs one complete inspection from start to finish, in the background so the window stays responsive:

1. classify the part (trained YOLO model, Gemini, or the offline OpenCV rules)
2. measure it (`opencv_measurement.py`)
3. check tolerances and pick a bin (`tolerance_and_bin_decision.py`)
4. save the result (`sqlite_database.py`)
5. start the physical sorting cycle (`esp32_communication.py`)

MAIN PARTS

- `FastenerInspectionWorker`: the background thread that performs the five steps above
- Sort-by-type mode: step 3 only picks the bin for the fastener type; sizes are shown but not judged
- Shape cross-check: a 'washer' whose outline has corners is corrected to a nut
- `FastenerClassifierManager`: what the UI calls to start an inspection; also keeps the batch counters

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **INSPECTION WORKER** - Runs one full inspection in a background thread: classify -> measure -> verify -> save -> sort.
- **INSPECTION MANAGER** - Starts inspections for the UI and keeps the batch counters.

Uses: backend/app_config.py, backend/app_logging.py, backend/esp32_communication.py, backend/gemini_cloud_classifier.py, backend/opencv_measurement.py, backend/opencv_shape_classifier.py, backend/sqlite_database.py, backend/tolerance_and_bin_decision.py, backend/yolo_classifier.py

Used by: frontend/main_window.py

backend/esp32_communication.py

Talks to the ESP32 that drives the machine (pad servo, chute servo, conveyor stepper). Commands are sent as JSON text over USB serial or Wi-Fi. With no hardware connected, a built-in simulator stands in for the ESP32 so the app can still be demonstrated.

MAIN PARTS

- Emergency stop: latch, block motion commands, reset
- Connection: Simulator / USB Serial / Wi-Fi
- `send_command()`: deliver one JSON command
- Sorting cycle: chute -> tilt pad -> conveyor -> release -> reset
- Simulator: pretends to be the ESP32
- `hardware_manager`: the shared hardware object the rest of the app uses

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **HARDWARE MANAGER** - One object that talks to the ESP32 (or to the built-in simulator).
 - **EMERGENCY STOP** - Latch the stop, block motion commands while it is active, and reset it.
 - **CONNECTION** - Choose Simulator / USB Serial / Wi-Fi, then connect or disconnect.
 - **SEND COMMAND** - Deliver one JSON command to the ESP32.
 - **SORTING CYCLE** - Chute -> tilt pad -> conveyor -> release -> reset, run in a background thread.
 - **SIMULATOR** - Pretends to be the ESP32 when no hardware is connected.
 - **SERIAL PORT LIST**
- **SHARED INSTANCE** - Created once here; every other file imports this same object.

Uses: backend/app_logging.py, backend/sqlite_database.py

Used by: backend/inspection_pipeline.py, frontend/main_window.py, frontend/tab_bins_and_chute_angles.py, frontend/tab_hardware_control.py, run_all_tests.py

Frontend - windows, tabs and buttons

frontend/main_window.py

The main application window: the header bar, the six tabs and the status bar. It also owns Tab 1 (Live Inspection) and connects the buttons to the backend.

MAIN PARTS

- Header: bins summary, camera selector, Settings button, EMERGENCY STOP
- Tab 1 Live Inspection: image view, Open File / Load Sample / Live Video, the INSPECT button, and the 'Sort by Type Only' option (bin chosen from the fastener type; size not judged)
- 'Save Photo for Training': saves the camera picture, labelled, for retraining the YOLO model
- Tabs 2-6 are built in their own files (tab_*.py)
- Camera handling, running an inspection, showing the result, emergency stop

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **WINDOW CONSTANTS**
- **MAIN WINDOW**
 - **STARTUP** - Create the backend engines, build the UI and connect the signals.
 - **WINDOW SIZING**
 - **LAYOUT** - Header bar, the six tabs and the status bar.
 - **TAB 1 LAYOUT** - Image view, toolbar buttons and the result panel.
 - **HEADER HELPERS** - Bins summary, camera list and the sample menu.
 - **IMAGE SOURCE** - Open an image file or load a built-in sample.
 - **LIVE CAMERA** - Start video, receive frames, freeze a frame, stop.
 - **TRAINING PHOTOS** - Save the current camera picture into my_camera_photos/<CLASS>/ for retraining the YOLO model.
 - **IMAGE DISPLAY**
 - **RUN AN INSPECTION** - What happens when the big INSPECT button is pressed.
 - **INSPECTION RESULTS** - Update the result panel, history, bin fill levels and counters.
 - **AI SETTINGS**
 - **EMERGENCY STOP BUTTON**
 - **SHUTDOWN**

Uses: backend/app_config.py, backend/app_logging.py, backend/camera_capture.py, backend/esp32_communication.py, backend/image_loading_and_overlays.py, backend/inspection_pipeline.py, backend/opencv_measurement.py, backend/opencv_shape_classifier.py, backend/sqlite_database.py, backend/tolerance_and_bin_decision.py, backend/yolo_classifier.py, frontend/ai_settings_dialog.py, frontend/app_stylesheets.py, frontend/inspection_result_panel.py, frontend/tab_bins_and_chute_angles.py, frontend/tab_camera_calibration.py, frontend/tab_hardware_control.py, frontend/tab_inspection_history.py, frontend/tab_iso_specifications.py

Used by: main.py, run_all_tests.py

frontend/inspection_result_panel.py

The panel on the right of Tab 1 that shows the result of the last inspection.

MAIN PARTS

- Category and decision badge (ACCEPT / REINSPECT / REJECT / IDENTIFIED / NOT RECOGNIZED)
- Four measurement tiles (length, shank diameter, head width, hole diameter)
- Destination bin and chute angle gauge
- Reason text
- Batch statistics and the reset button

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **RESULT PANEL**
 - **LAYOUT** - Outcome header, measurement tiles, chute destination, notes, batch statistics.
 - **UPDATE THE PANEL** - Show a finished inspection, refresh the counters, or show 'analyzing'.

Uses: backend/app_config.py

Used by: frontend/main_window.py

frontend/tab_bins_and_chute_angles.py

Tab 2 'Custom Bins & Chute Angles': set how many sorting bins the machine has and, for each bin, which fastener it collects, its chute servo angle and its capacity.

MAIN PARTS

- Quick setup: number of bins and automatic angle spacing
- Bins table: one editable row per bin, with fill level and a 'test chute' button
- Save to database, reset counts

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **BINS TAB**
 - **LAYOUT** - Quick setup row, the bins table and the bottom buttons.
 - **BINS TABLE** - Build one editable row per bin from the database.
 - **BUTTON ACTIONS** - Auto-space the angles, save, test a chute position, reset counts.

Uses: backend/app_logging.py, backend/esp32_communication.py, backend/sqlite_database.py

Used by: frontend/main_window.py

frontend/tab_hardware_control.py

Tab 3 'Hardware & Sorting Chute': choose how to connect to the ESP32, watch its live status, and move each motor by hand for testing.

MAIN PARTS

- Connection bar: Simulator / USB Serial / Wi-Fi
- Live telemetry: state, servo angles, conveyor, battery
- Manual tests: tilt pad, run conveyor, move chute

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **HARDWARE TAB**
 - **LAYOUT** - Connection bar, live telemetry grid and manual test buttons.
 - **BUTTON ACTIONS**
 - **LIVE UPDATES** - Called whenever the hardware manager reports new status.

Uses: backend/app_logging.py, backend/esp32_communication.py, backend/sqlite_database.py

Used by: frontend/main_window.py

frontend/tab_iso_specifications.py

Tab 4 'ISO Specifications': view, add and delete the standard fastener sizes and their tolerance limits. These are the limits a measured part is checked against.

MAIN PARTS

- AddSpecificationDialog: the form for adding a new size
- SpecificationPanel: one table per category (Bolts, Nuts, Washers, Screws...)

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **ADD-SIZE DIALOG** - The form for adding a new fastener size.
- **SPECIFICATIONS TAB** - One table of sizes per fastener category.
 - **TABLE CONTENT AND BUTTON ACTIONS**

Uses: backend/app_logging.py, backend/sqlite_database.py

Used by: frontend/main_window.py

frontend/tab_camera_calibration.py

Tab 5 'Camera Calibration': set the camera scale (millimetres per pixel) that converts pixel measurements into real sizes.

MAIN PARTS

- Calibrate from a part: inspect a part, type its real size measured by hand, and the scale is worked out
- Manual scale: type the millimetres-per-pixel value directly
- Save to database / reset to default

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **CALIBRATION TAB**
 - **LAYOUT**
 - **CALIBRATE FROM A PART** - new scale = old scale x (real size / size the app measured)
 - **BUTTON ACTIONS** - Save or reset the millimetres-per-pixel scale.

Uses: backend/app_logging.py, backend/opencv_measurement.py, backend/sqlite_database.py

Used by: frontend/main_window.py

frontend/tab_inspection_history.py

Tab 6 'Quality Audit History': a table of every inspected part with its measurements and decision.

MAIN PARTS

- Loads saved records from the database when the app starts
- Adds a row after each new inspection
- Clear history, export to CSV

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **HISTORY TAB**
 - **LAYOUT**
 - **TABLE CONTENT** - Load saved records and add a row for each new inspection.
 - **BUTTON ACTIONS** - Clear the history or export it to a CSV file.

Uses: backend/app_config.py, backend/app_logging.py, backend/sqlite_database.py

Used by: frontend/main_window.py

frontend/ai_settings_dialog.py

The Settings window (opened from the Settings button in the header): choose the AI model and enter the Gemini API key.

MAIN PARTS

- ConnectionTestWorker: tests the API key in the background
- SettingsDialog: the window itself
- Saving writes GEMINI_API_KEY and GEMINI_MODEL to the .env file

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **BACKGROUND CONNECTION TEST**
- **SETTINGS DIALOG**
 - **BUTTON ACTIONS** - Show or hide the key, test the connection, save to .env.

Uses: backend/app_config.py, backend/app_logging.py, backend/gemini_cloud_classifier.py, backend/yolo_classifier.py

Used by: frontend/main_window.py

frontend/app_stylesheet.py

The visual style of the whole app (colours, fonts, borders, button looks), written as one Qt stylesheet. Change the look here; nothing in this file affects behaviour.

MAIN PARTS

- MAIN_STYLESHEET: sections are marked with /* ... */ comments (cards, buttons, inputs, tabs, tables, progress bars, status bar)

Used by: frontend/main_window.py

Tools - run from the command line

build_classifier_dataset.py

Turns the downloaded Roboflow dataset ("FASTENER SORTER.v3i.yolov11": whole photos with a box drawn around each part) into the folder-per-class layout that train_yolo_classifier.py needs:

fastener_sorter_cls/train/<CLASS>/... val/<CLASS>/... test/<CLASS>/...

Run it again whenever the downloaded dataset changes:

python build_classifier_dataset.py

MAIN PARTS

- Part crops: every labelled part is cut out three times (tight, medium, wide), so the model also learns to recognise a part that does not fill the picture, as happens with a hand-held camera
- Whole photos: photos showing a single kind of part are also used uncropped
- NO_FASTENER class: empty patches of background plus plain frames, so the model can say "nothing here" instead of always naming a fastener
- Your own photos: everything saved with the app's "Save Photo for Training" button (my_camera_photos/<CLASS>/) is added, so the model learns your camera, parts and lighting
- Extra photos: ready-made pictures of single parts in extra_training_photos/<CLASS>/ are added whole

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **READING THE DOWNLOADED LABELS**
- **PART CROPS AND WHOLE PHOTOS**
- **NO_FASTENER CLASS** - Background patches that touch no labelled part, plus plain frames.
- **YOUR OWN CAMERA PHOTOS** - Each photo is used whole and as the centre guide box (where the part is held during live video).
- **RUN**

train_yolo_classifier.py

Trains the YOLO11n-cls model that recognises the fastener type from a picture, then tests it and saves the finished model where the app looks for it. Run it whenever the dataset changes:

```
python train_yolo_classifier.py
```

Training runs on the CPU. The app itself never trains; it only loads the saved model.

MAIN PARTS

- Train: start from the pretrained yolo11n-cls weights and train on <dataset>/train, checking on <dataset>/val
- Save: copy the best weights to models/fastener_yolo11n_cls.pt and the class list next to it
- Test: accuracy, per-class precision / recall / F1 and the confusion matrix on <dataset>/test

SECTIONS INSIDE THE FILE (the headings you will see in the code)

- **TRAIN** - Fine-tune the pretrained YOLO11n classification model on the fastener images.
- **TEST** - Score the saved model on images it has never seen (the test split).
- **RUN**

run_all_tests.py

Automated check that the whole system still works. Run it after changing code:

```
python run_all_tests.py
```

It uses a temporary database, so real specifications, bins and counters are not touched.

MAIN PARTS

- 1. Database and saved counters
- 2. Emergency stop
- 3. Sample image generator
- 4. OpenCV classifier + measurement on the sample images
- 5. Tolerance check and bin decision
- 6. ESP32 simulator
- 7. Main window opens with all six tabs
- 8. Trained YOLO classifier (skipped when no model has been trained)

Uses: backend/app_config.py, backend/app_logging.py, backend/esp32_communication.py, backend/image_loading_and_overlays.py, backend/opencv_measurement.py, backend/opencv_shape_classifier.py, backend/sqlite_database.py, backend/tolerance_and_bin_decision.py, backend/yolo_classifier.py, frontend/main_window.py

4. ESP32 firmware

firmware_esp32/firmware_esp32.ino

The Arduino program that runs on the ESP32 board inside the machine. It listens for commands from the laptop over USB serial (115200 baud) or over its own Wi-Fi network (TCP port 8080), moves the motors, and replies. Each command and reply is one line of JSON text.

HARDWARE IT DRIVES

- Inspection pad tilt servo (MG996R) on pin 18
- Rotating chute servo (MG996R) on pin 19
- Conveyor stepper (NEMA 17 through a TB6600 driver): STEP 22, DIR 23, ENABLE 21
- Two IR sensors: part on pad (pin 34) and part on conveyor (pin 35)

COMMANDS IT UNDERSTANDS

Command	What the ESP32 does
SORT (tray, angle)	Turns the chute servo to the given angle.
TILT_PAD (angle)	Tilts the inspection pad to drop the part onto the conveyor.
CONVEYOR (RUN / stop)	Runs the conveyor stepper for a set time, or stops it.
RESET	Returns pad and chute to their home positions.
ESTOP / RESET_ESTOP	Emergency stop: cuts motion and refuses other commands until reset.
PING / STATUS	Replies with the current state (angles, conveyor, connection).

SECTIONS INSIDE THE FILE

- **PIN NUMBERS** - Which ESP32 pin each servo, driver and sensor is wired to.
- **WI-FI ACCESS POINT** - Network name, password and TCP port the laptop connects to.
- **CURRENT MACHINE STATE**
- **SETUP** - Runs once at power-on: pins, servos, home positions, Wi-Fi.
- **MAIN LOOP** - Runs forever: wait for a command from USB serial or Wi-Fi.
- **COMMAND HANDLER** - Decode one JSON command and act on it.
- **CONVEYOR** - Step the NEMA 17 motor for a fixed time.
- **REPLIES TO THE LAPTOP** - Acknowledgement after a command, and the telemetry status message.

Talks to: backend/esp32_communication.py on the laptop.

5. How to run it

Command (run inside the project folder)	What it does
pip install -r requirements.txt	Installs the Python packages. Needed once on each computer.
python main.py	Opens the application.
python run_all_tests.py	Runs the 8 automatic checks. Uses a temporary database, so real data is not touched.
python build_classifier_dataset.py	Rebuilds the training images from the Roboflow download and your saved camera photos.
python train_yolo_classifier.py	Retrains the YOLO model and writes its accuracy report into models/. Takes about 35 minutes on a laptop CPU.

The firmware is opened in the Arduino IDE and uploaded to the ESP32. It needs the ESP32Servo and ArduinoJson libraries. The application also works with no ESP32 connected: Tab 3 has a built-in simulator.